PROJECT PLANNING DOCUMENT — V1.0 · DRAFT

FORJA

CODE THAT STAYS IN SPEC.

An AI-assisted automation engineering platform — turning functional specs into compiling Siemens TIA Portal projects.

CLIENT	AUTHOR	DATE	STATUS
Pac Technologies	Kasper Simonsen	2026-04-29	Draft

ENGAGEMENT	STACK	PAGES	DOCUMENT ID
Internal tooling	React 19 · Supabase · Claude	29	ks-ppd-forja-v1

hello@kaspersimonsen.dev

kaspersimonsen.dev / Melbourne · AEST · UTC+10

00 · CONTENTS

22 SECTIONS

01	Executive Summary	BRIEF
02	Project Brief	BRIEF
03	Users & Personas	DISCOVERY
04	Engineer Journey Map	DISCOVERY
05	Product Scope	SCOPE
06	Module Specifications	SCOPE
07	Cross-Module Integration	ARCHITECTURE
08	Non-Functional Requirements	ARCHITECTURE
09	Technical Architecture	ARCHITECTURE
10	Data Model Summary	ARCHITECTURE
11	Third-Party Dependencies	RISK
12	Hosting & Deployment Plan	DELIVERY
13	Security, Privacy & IP	RISK
14	Testing & Validation	DELIVERY
15	Roadmap & Milestones	DELIVERY
16	Resource Plan	DELIVERY
17	Risk Register	RISK
18	Assumptions & Open Questions	RISK
A	Glossary	APPENDIX
B	Out of Scope (MVP)	APPENDIX

01 BRIEF

Executive summary

Forja is an AI-assisted automation engineering platform that compresses the time required to design, specify, and code a Siemens TIA Portal project. It pairs a structured functional-spec authoring environment — the **FDS Builder** — with a guided code-generation wizard — the **Forge Wizard** — backed by a continuously-learning pattern library and a catalogue of pre-built, version-controlled function block templates.

Built initially as an internal tool for Pac Technologies engineers, the goal is to allow a single engineer to run multiple projects in parallel — turning the studio's capacity constraint from engineer-hours per project into projects per engineer. A licensing path to external automation firms is a secondary, longer-horizon objective once the tool is proven internally.

The MVP delivers four integrated modules: structured spec authoring, a fifteen-step code-generation wizard, a continuously-learning correction pattern library, and a versioned catalogue of pre-built function block templates. HMI screen generation, third-party robot vendor integration, and external licensing are explicitly out of scope for the MVP — see Appendix B.

02 BRIEF

Project brief

Background

Pac Technologies delivers Siemens TIA Portal automation projects to industrial customers across Australia and New Zealand. A typical mid-sized project takes a senior engineer four to eight weeks — moving from a customer's functional brief through device specification, code authoring, HMI development, simulation testing, and on-site commissioning.

Most of that time is repetitive scaffolding. The same patterns of conveyor logic, motor sequencing, and alarm handling get written, by hand, project after project.

Modern AI tools can generate plausible PLC code from a prompt, but they make systematic mistakes the moment the project deviates from common patterns: incorrect IO mapping, missed safety interlocks, inconsistent naming, code that compiles but doesn't behave correctly. A general-purpose AI is not enough.

The engineering knowledge embedded in a senior engineer's head — house style, customer conventions, vendor quirks, hard-won corrections — needs a place to live and a way to influence every generation. Forja is that place.

Goals

BUSINESS

- Compress the engineer-hours per project by 3× or more on representative work.
- Run multiple concurrent projects without losing code quality.
- Capture institutional knowledge in a form that survives staff turnover.
- Establish a foundation that can be commercialised externally — without rebuilding from scratch.

ENGINEERING

- Reduce the cognitive load of switching between projects.
- Eliminate the most common categories of repeated rework — naming, IO wiring, missed permissives.
- Produce TIA-importable code that compiles first attempt for at least 80% of generated artefacts.
- Make the engineer's review the primary act of value, not the typing.

SUCCESS METRICS — THREE MONTHS POST-LAUNCH

METRIC	TARGET
Active engineer-users	2-3
Projects delivered using Forja end-to-end	≥ 3
Engineer-hour reduction per project (vs baseline)	≥ 50%
Generated SCL artefacts that compile first attempt	≥ 80%
Approved correction patterns in the library	≥ 100
Function block templates in the assembly catalogue	≥ 8 (V1)
Reference documents indexed	≥ 5
Critical regressions per week post-launch	< 2

Scope · MVP

IN SCOPE

- **FDS Builder** — structured functional spec authoring environment
- **Forge Wizard** — fifteen-step guided TIA Portal code generation
- **Pattern Librarian** — continuous learning system that captures and re-uses correction patterns
- **Assembly FB Library** — versioned catalogue of pre-built, parameterised function block templates
- **Reference Library** — uploadable Siemens documentation indexed for AI retrieval at generation time
- **TIA Portal Openness Bridge** — Windows-side integration for project import, compile, and export
- Internal authentication and per-engineer state persistence

NOT CURRENTLY TAKING ON

Tracked for future phases. See Appendix B for the rationale on each.

- HMI screen generation — separate workstream
- Third-party vendor integration — KUKA robots, ABB drives, custom OEM components
- Automated unit-test generation
- External licensing infrastructure — multi-tenancy, billing, customer onboarding
- iOS / Android engineering apps
- Live customer-site monitoring or runtime telemetry

Constraints

Budget	Internal initiative — no external development spend. Operating costs (Supabase, Anthropic API, Vercel) budgeted separately.
Timeline	First useful version targeted within 6 months. Internal MVP at month 6, internal pilot at months 6–9, optional Vercel migration at month 9.
Team	One full-time engineer-developer (Kasper). No external contractors planned.
Platform	Siemens TIA Portal V17–V20 only. Windows-only for the TIA Bridge — a hard constraint of TIA Openness. Frontend is browser-based, cross-platform.
Regulatory	No safety certification claimed for AI-generated code at MVP. All generated artefacts pass through engineer review before deployment to customer sites. Non-negotiable.

03 DISCOVERY

Users & personas

Three personas, in priority order.

PRIMARY USER — 01

Senior automation engineer

- Ten-plus years on TIA Portal, fluent in SCL and LAD.
- Currently runs one or two concurrent projects; wants to run three or four without dropping quality.
- Has strong opinions about house style and naming conventions.
- Sceptical of AI tools that produce plausible but subtly wrong code.
- Wants the tool to disappear when it's working and surface clearly when it isn't.
- Will not adopt anything that costs more time than it saves on day one.

> *"If I'm correcting the same mistake twice, the tool isn't doing its job."*

SPONSOR — 02

Studio principal

- Sets project pricing and capacity targets.
- Watches engineer utilisation and project margin.
- Cares about Forja's effect on throughput and quality, not its internals.
- Holds the budget for ongoing operational costs.
- Measures success by projects delivered and customer satisfaction — not by lines of code generated.

SECONDARY — 03 — POST-MVP

Junior automation engineer

- Two to four years' experience.
- Currently restricted to simpler projects or works under supervision.
- Forja could let them deliver projects independently sooner — with the tool encoding the senior's review patterns.
- Not a primary MVP user. Their needs shape Phase 2 design.

JOBS TO BE DONE

What the engineer is trying to accomplish

PRIMARY

When I receive a functional brief from a customer, I want to author a structured functional specification in a form that drives downstream code generation — so I produce executable engineering rather than prose that has to be re-interpreted later.

When I have a structured spec, I want to generate a complete TIA Portal project that compiles on first import — so my time is spent on review and refinement rather than typing boilerplate.

When the AI makes a mistake I have to correct, I want that correction captured and applied to all future generations — so I never make the same fix twice.

SECONDARY

When I'm running multiple projects, I want each one to remember exactly where I left off — so context-switching costs are minimised.

When I encounter a structurally familiar machine — a conveyor, a pusher, a lift table — I want the tool to use a proven template rather than re-generate from scratch — so output quality is dictated by my best work, not the AI's average.

When I'm uncertain about a Siemens instruction or convention, I want the tool to retrieve the relevant section of the official documentation and use it — so I don't have to break flow to look things up.

04 DISCOVERY

Engineer journey map

End-to-end journey for a senior engineer on a representative project — a five-station conveyor and lift line.

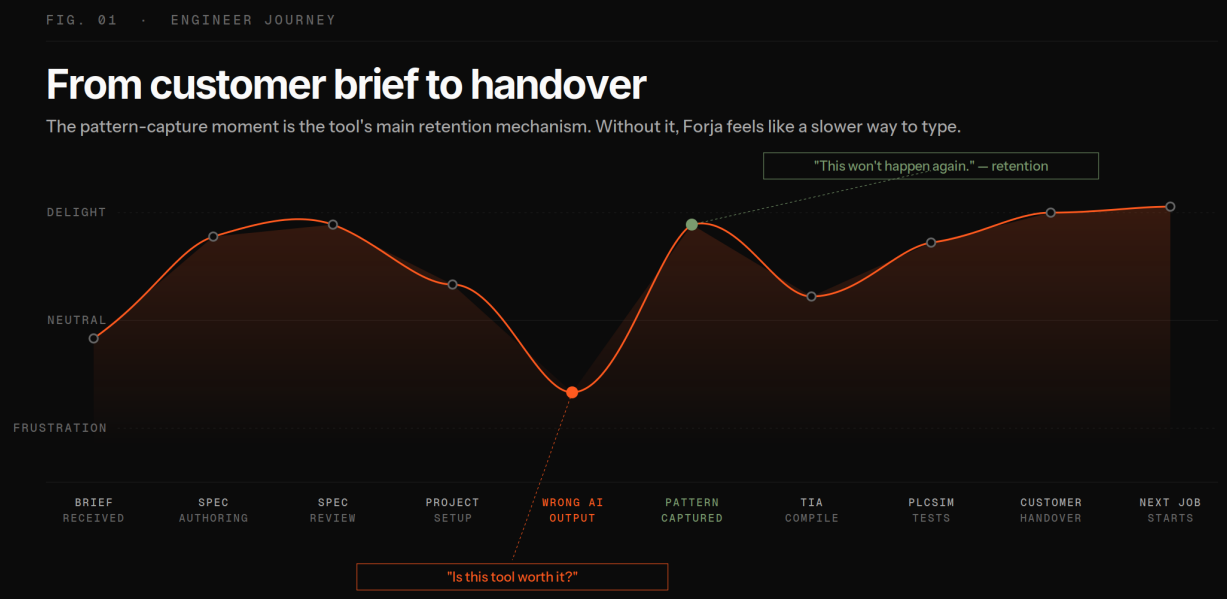

FIG. 01 — ENGINEER EMOTIONAL JOURNEY THROUGH A TYPICAL FORJA-ASSISTED PROJECT

STAGE	ACTION	EMOTION	TOUCHPOINT
Brief received	Customer sends a functional brief and IO list	Mild fatigue	–
Spec authoring	Uploads IO register, defines machine hierarchy, walks assembly-by-assembly behavioural review	Engaged	FDS Builder
Spec review	Reviews the composed spec, generates DOCX for customer sign-off	Satisfied	FDS Builder export
Project setup	Links the approved spec to a new Forge session	Relief	Wizard 01-03
Hardware & IO	Confirms rack/module config, reviews AI-matched device templates	Confident	Wizard 04
Code generation		Mixed	Wizard 06-12

STAGE	ACTION	EMOTION	TOUCHPOINT
	Generates device FBs, assembly FBs, process logic		
Review & correct	Edits the wrong artefact; correction is captured as a pattern	Quiet satisfaction	Pattern Librarian
TIA upload	Bridge pushes to TIA, compile runs, errors surface in chat	Focused	TIA Bridge · compile-fix
PLCSim	Runs simulation tests	Confident	Wizard 11, 14
Handover	Exports the project, sends to customer, archives session	Pride	Wizard 15

> DESIGN INSIGHT
> The pattern-capture moment is the tool's main retention mechanism. If the engineer doesn't see the learning happen, the tool feels like a worse version of writing it themselves.

05 SCOPE

Product scope

Five integrated modules in the MVP. Each is described in detail in §06.

MODULE	PURPOSE	MVP SCOPE
FDS Builder	Author structured functional specifications	All six phases — instrument register, hierarchy wizard, co-authoring, system orchestration, structured editor, DOCX export
Forge Wizard	Generate TIA Portal projects from a spec	All fifteen steps, except HMI generation (Step 13 stubbed)
Pattern Librarian	Capture and re-use correction patterns	Auto-capture on rewrite, compile-fix, manual fix; admin review queue
Assembly FB Library	Catalogue of pre-built FB templates	V1 catalogue of 8 templates with full interface contracts
Reference Library	Indexed authoritative documentation	Upload + section + tag, two-pass AI lookup at generation time

06 SCOPE

Module specifications

Feature definition with acceptance criteria. Implementation detail lives in the technical architecture and per-module engineering notes alongside the codebase.

06.1

FDS Builder — Functional Description Specification authoring

The FDS Builder is the engineering side of Forja. It produces the same functional specification document a senior engineer would write by hand — but as structured, machine-readable data rather than prose.

PHASES

01 Instrument Register Upload — upload an IO spreadsheet (CSV or Excel). The system classifies each tag and stores it as the authoritative tag source for all subsequent phases.

02 Spec Skeleton Wizard — six-step guided form to define the machine hierarchy: System / Subsystem / Assembly / Device, plus operating states and alarm tiers.

03 FDS Co-Authoring — per-assembly behavioural specification, conducted as a structured AI interview. Static device states are auto-filled deterministically; sequential states are co-authored with an AI agent.

04 System Orchestration — subsystem-to-subsystem coordination: ordering, shared permissives, inter-subsystem interlocks.

05 Structured Spec Editor — document tree navigation, section-by-section approval flow.

06 DOCX Export — formatted Word output in the Pac Technologies house style.

ACCEPTANCE CRITERIA

- An engineer can produce a complete behavioural specification for a 5-station, 25-device machine in under a working day.
- The spec is the authoritative source of truth for downstream code generation — no re-interpretation.
- For structurally identical assemblies, behavioural data can be cloned and tag-remapped automatically.
- DOCX output is presentable to a customer without manual reformatting.

06.2

Forge Wizard — TIA Portal code generation

The coding side of Forja. Takes a functional specification (from the FDS Builder, or a manually uploaded document) and runs it through a fifteen-step pipeline that ends with a complete, compiled TIA Portal project.

STEPS

#	NAME	OUTPUT
01	Spec	Auto-loaded from FDS Builder, or manual document upload
02	Q&A Review	Clarifying interview by AI Project Manager
03	Project Setup	CPU, TIA version, design profile, language preferences
04	Hardware & IO	Rack and module config; AI device template matching
05	Interface Contracts	Per-assembly exposed/consumed signals
06	Device FBs	Per-device function block — library match preferred
07	Assembly FBs	Per-assembly state machine — library match preferred
08	Logic Check	Deterministic validation against FDS spec across 10 categories
09	Matrix Review	Device linkage matrix and process sequence generation
10	Device Code	Device call FCs, IO linking FC, OB1 main scaffold
11	Device Tests	PLCSim-based tests for individual device FBs
12	Process Code	Process-layer FCs, step/action DBs, fault handling
13	HMI Screens	Deferred — stubbed in MVP
14	System Tests	PLCSim-based full-program tests
15	TIA Export	Final import and compile in TIA Portal

ACCEPTANCE CRITERIA

- A wizard session can be paused and resumed at any step without state loss.
- Each step exposes its outputs for engineer review before the next step proceeds.
- Generation is parameterised by the engineer's chosen design profile — naming, language, conventions.

- Library-matched artefacts are copied verbatim with no AI involvement; AI generation is a fallback for novel cases.
- The deterministic logic checker (Step 08) provides actionable, FDS-grounded findings — not generic advice.

06.3

Pattern Librarian — Continuous learning

Every code correction in Forja — whether from compile-fix, manual TIA fix, pipeline review, or engineer chat — is captured as a structured WRONG/CORRECT pattern. Approved patterns are injected into every future generation prompt so the same mistake is never repeated.

CAPTURE SOURCES

- Forge Wizard rewrite loop (per artefact)
- Forge Wizard compile-fix cycles — auto-approved on successful compile
- Manual TIA fix panel
- Direct engineer teaching via chat
- Document-extraction teaching dialogs

CLASSIFICATION

Patterns are classified into one of six types — `NAMING` · `IO_MAPPING` · `STATE_LOGIC` · `ALARM` · `SAFETY` · `TIMING`. AI classification is the primary path; a regex-based classifier is the deterministic fallback.

LIFECYCLE

`PENDING` — `APPROVED` or `REJECTED`, with revoke and delete actions. Only approved patterns are injected into future generation prompts.

ACCEPTANCE CRITERIA

- Pattern capture is fire-and-forget — never blocks the main generation flow.
- Admin review queue defaults to PENDING — engineers see new patterns first.
- Patterns are scoped to a PLC brand (currently SIEMENS_TIA), allowing future expansion without contamination.
- An approved pattern visibly affects future generations — verifiable by re-running a previously failed generation.

06.4

Assembly FB Library — Versioned function block catalogue

Replaces greenfield AI-generated assembly code with a catalogue of typed, versioned, instance-parameterised templates. When a spec contains an assembly of a known type, Forja uses the library template directly. AI generation becomes the fallback for novel assembly types.

V1 CATALOGUE — 8 TEMPLATES

TEMPLATE	DESCRIPTION
`ConveyorStandardVsd`	Conveyor with variable speed drive
`ConveyorStandardDol`	Conveyor with direct-on-line motor
`TransferTable2Axis`	Two-axis transfer table
`TurntableSingleStop`	Single-stop turntable
`PusherLinearCylinder`	Linear pusher cylinder
`DiverterSwingGate`	Swing-gate diverter
`LiftStationVertical`	Vertical lift station
`AccumulatorBufferConveyor`	Buffer accumulator conveyor

ACCEPTANCE CRITERIA

- A new template can be authored in under a working day by a senior engineer.
- A spec assembly that matches a template is wired automatically; the engineer confirms or overrides.
- Library template versions are pinned at the time of binding — later library edits do not silently retrofit existing specs.
- Each template's body SCL is validated by compiling in TIA Portal V18 before being marked production-ready.

06.5

Reference Library — Indexed authoritative documentation

Engineers upload Siemens manuals, instruction references, style guides, and customer conventions. Forja splits each document into searchable sections with topic tags. At generation time, a two-pass AI retrieval pulls the most relevant sections into the prompt as authoritative context.

TWO-PASS RETRIEVAL

01 The user's request — or generated code, or compile errors — is analysed by AI to extract relevant topic keywords.

02 A Postgres full-text + tag-overlap search returns the matching reference sections.

03 The sections are formatted and injected into the system prompt for the generation step.

ACCEPTANCE CRITERIA

- A 500-page manual can be uploaded and indexed in under fifteen minutes.
- Engineers can browse, search, and tag sections through a UI.
- Reference results are filtered by programming language (LAD / SCL / general) — an SCL generation doesn't get LAD documentation.
- All four AI generation paths consume the reference library uniformly.

07 ARCHITECTURE

Cross-module integration

Forja's modules are loosely coupled and clearly bounded. The integration map below documents how data flows between them.

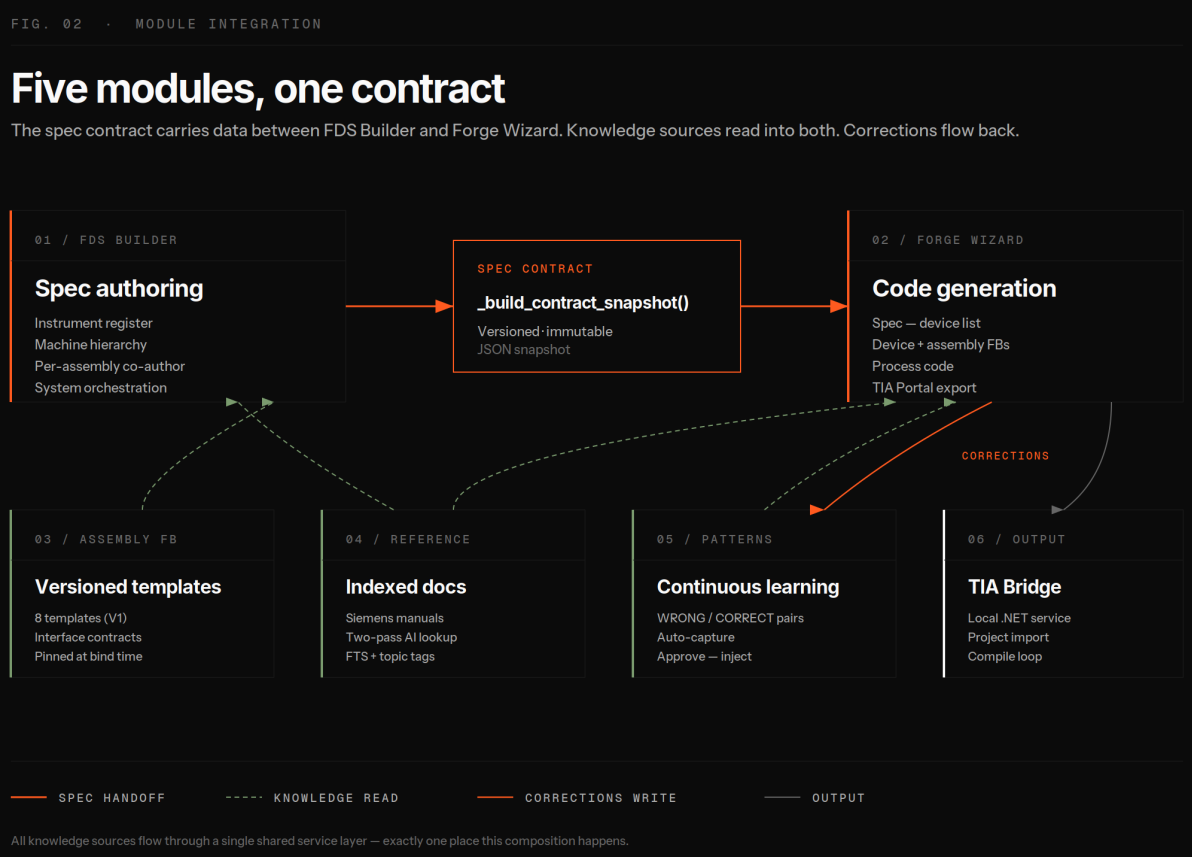

FIG. 02 — MODULE INTEGRATION. SPEC DATA FLOWS ORANGE; KNOWLEDGE INJECTION FLOWS GREEN; CORRECTIONS FLOW BACK.

> **THE HANDOFF THAT MATTERS MOST**
>
> The handoff from FDS Builder to Forge Wizard is the single most important integration in the system. It's implemented as a Postgres function — `_build_contract_snapshot()` — that returns a complete spec contract as a JSON snapshot.
>
> Forge Wizard reads from the snapshot, never from the FDS Builder's internal tables. This boundary protects both modules from one another's internal evolution.

The Reference Library and Pattern Librarian are read-only services consumed by all generation paths uniformly through a single shared service layer. This is deliberately designed to avoid the trap of "four

generation paths each with their own context-building logic" — a known anti-pattern in AI-driven systems.

The Assembly FB Library is consumed at two points — at FDS Builder time (template selection per assembly during the hierarchy phase) and at Forge Wizard time (artefact emission during assembly FB generation). Both consumption points read from the same `fb_templates` table.

08 ARCHITECTURE

Non-functional requirements

Performance	Wizard step transitions under 500ms. AI generation latency dominated by model response time; UI must remain responsive throughout.
Availability	99% target for internal use during business hours. Planned maintenance permitted with 24-hour notice.
Concurrency	Support 3 simultaneous engineer-users at MVP, with clean session isolation. Architected to scale to 20+ without redesign.
Security	Supabase auth with per-engineer accounts. Row-level security on all engineer-scoped tables. No customer project data leaves the studio's infrastructure.
Data integrity	All structured artefacts (specs, FBs, patterns) are versioned. Destructive operations require explicit confirmation.
Auditability	Every AI generation persists its full prompt, response, model, and token count. Every approved pattern records its source.
Reproducibility	A spec contract snapshot, plus the pinned library and pattern versions, is sufficient to re-run a generation deterministically.
Cost ceiling	AI provider spend monitored monthly. Per-engineer alerts at $200/month, hard cap at $500/month at MVP scale.
Browser support	Latest two versions of Chrome and Edge. No mobile support targeted at MVP.
Offline behaviour	Read-only access to existing specs and artefacts when AI provider is unreachable. Generation requires connection.

09 ARCHITECTURE

Technical architecture

Stack

LAYER	CHOICE	RATIONALE
Frontend	React 19 + Vite + TypeScript	Modern, fast iteration; strong typing essential for a structured-data-heavy app
UI	shadcn/ui (Radix) + Tailwind	Engineer-tool aesthetic — dense, dark-first, no airy SaaS look
State	TanStack Query + Zustand	Server state via Query, UI state via Zustand. Avoids the trap of one giant store.
Editor	Monaco Editor	Industry-standard, custom SCL language definition
Backend	Supabase	Postgres + Edge Functions + Auth in one. Minimises operational surface for a solo developer.
AI provider	Anthropic Claude	Sonnet for generation and review; Haiku for classification and tag inference
TIA bridge	.NET Framework 4.8 console app	Required by TIA Openness; runs locally on the engineer's Windows machine
File storage	Dropbox (shared workspace)	Engineers already use it; avoids re-architecting team file flow
Hosting · initial	Local server inside the studio	Simplifies the early phase — no public exposure
Hosting · post-MVP	Vercel + Supabase	Standard, low-ops, scales for internal team size

System diagram

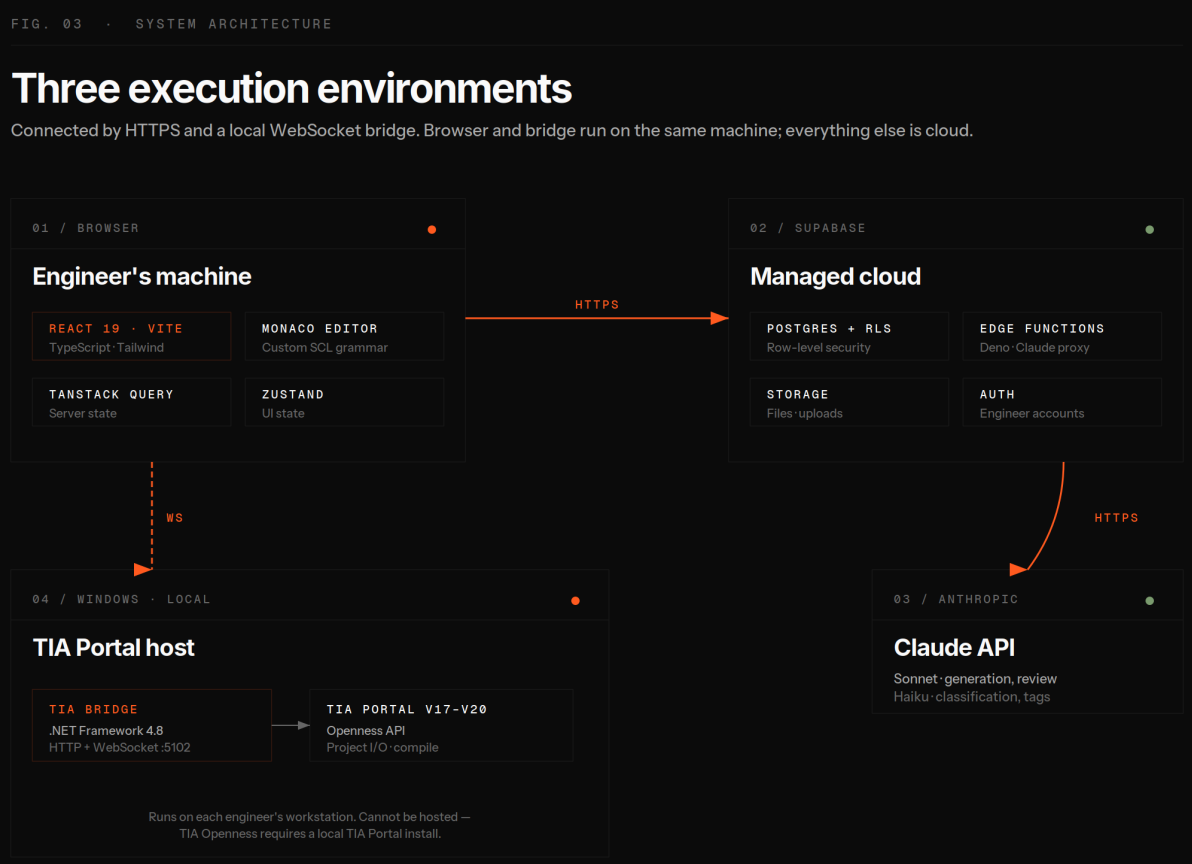

FIG. 03 — SYSTEM ARCHITECTURE. THREE EXECUTION ENVIRONMENTS CONNECTED BY HTTPS AND A LOCAL WEBSOCKET BRIDGE.

Architectural principles

Non-negotiable in MVP design and review.

01 **Module boundaries are honoured.** The five modules communicate through a small set of well-typed contracts. No module reaches across into another module's tables.

02 **Deterministic over generative wherever possible.** AI is used where structure cannot be derived; everything else is deterministic and auditable.

03 **Generation context is centralised.** All AI generation paths consume a single shared service that assembles patterns, knowledge, reference sections, design profiles, and platform rules.

04 **Spec contracts are immutable snapshots.** The handoff between modules is a versioned snapshot, not a live reference. This protects each module from the others' internal evolution.

05 **Every artefact is auditable.** Generated outputs carry their lineage — which prompt produced them, which patterns were active, which reference sections were retrieved.

06 **Engineer review is the value gate.** Forja generates; engineers approve. The tool never deploys directly to customer systems.

10 ARCHITECTURE

Data model summary

Six entity groups. Field-level detail lives in the migration files.

GROUP	TABLES / ENTITIES
Identity & auth	users, sessions, agent_leases
Specs	spec_projects, spec_sections, fds_assembly_sessions, fds_subsystem_orchestrations, fds_system_orchestrations, spec_alarms
Forge sessions	forge_sessions, forge_artifacts
Knowledge	fb_templates, fb_template_versions, pattern_candidates, reference_library_docs, reference_library_sections, agent_knowledge_docs, prompt_sections
Configuration	design_profiles, knowledge_priority_overrides
Audit	audit_log

TWO DESIGN CHOICES WORTH NOTING

- JSONB is used liberally for structured-but-evolving payloads — interface contracts, sequence steps, behavioural state tables. Each JSONB column has a documented schema and a TypeScript type derived from a Zod schema in the frontend.
- Spec contracts are assembled by a Postgres function — `_build_contract_snapshot` — rather than reconstructed in application code. This guarantees the FDS Builder's view of a spec and the Forge Wizard's view are identical.

11 RISK

Third-party dependencies

DEPENDENCY	PURPOSE	RISK PROFILE
TIA Openness	Bridge to TIA Portal for import, compile, export	High coupling, but stable across V17–V20. Each new TIA version requires bridge re-test.
Anthropic API	Primary AI provider	Cost monitored monthly. Pricing changes possible but historically gentle.
Supabase	Database, auth, edge functions, storage	Single-vendor risk; Postgres portability is a mitigation.
Dropbox	Shared file storage for design docs and library blocks	Already used by the studio; low integration risk.
Vercel	Frontend hosting (post-MVP)	Standard for Vite apps; trivially replaceable.
Monaco Editor	In-browser code editing	Microsoft-maintained, stable, widely used.
shadcn/ui	UI primitives	Source-included rather than vendored — no upgrade lock-in.

Future-phase dependencies — out of MVP scope:

- KUKA / ABB / vendor-specific protocol libraries — for third-party robot and drive integration. Deliberately deferred. The MVP design does not assume their absence will require structural changes.

12 DELIVERY

Hosting & deployment plan

The hosting plan evolves in three stages.

STAGE 01 — MONTHS 0-6

Local server

Forja is hosted on a single workstation inside the studio LAN. Supabase runs via Docker locally for development and against a hosted Supabase project for shared state. Minimises operational complexity while the tool is in active development.

STAGE 02 — MONTHS 6-9

Hosted Supabase, local frontend

Supabase moves to its hosted offering. The frontend continues to run from the engineer's machine via `npm run dev` for the initial pilot period. Validates the cloud database under real use before exposing the frontend.

STAGE 03 — MONTHS 9+

Vercel-hosted frontend

The frontend deploys to Vercel with environment-based access controls. The TIA Bridge continues to run locally on each engineer's Windows machine — this cannot be hosted, by design.

External-customer hosting is not addressed in this document and would require a separate architecture review — multi-tenancy, customer data isolation, and pricing model.

13 RISK

Security, privacy & IP protection

Customer project data

Customer functional specs may contain commercially sensitive information about the customer's process, equipment, and IP. Treatment of this data is non-negotiable.

- All customer project data is stored in Supabase under row-level security policies scoped to the originating engineer.
- Customer data is never shared with the AI provider beyond the immediate generation context, and is never used to train the AI provider's models. Anthropic's API terms support this — explicit verification at sign-up.
- Customer files (DOCX, PDF, Excel) are stored in Supabase Storage with signed URLs and a default expiry.
- Engineers can export and delete a project's full data on request.

Pac Technologies IP

The function block library, reference documents, correction patterns, design profiles, and prompt sections collectively constitute Pac Technologies' engineering IP.

- These artefacts live in Supabase, backed up daily.
- The catalogue is versioned — destructive changes are recoverable.
- A planned Phase 2 capability is "library export" — a portable bundle of the studio's IP that can be migrated, backed up offsite, or, eventually, licensed to external customers.

Authentication

Supabase email-and-password authentication for MVP. SSO via the studio's existing identity provider considered for a future phase if user count grows.

14 DELIVERY

Testing & validation strategy

Three layers of testing, in order of priority.

01 — AUTHORITATIVE

Compile validation against TIA Portal

The most authoritative test for generated code is whether TIA Portal accepts it. Every Forge Wizard run ends with an automated compile pass. Compile failures surface and route into the compile-fix loop.

02 — COVERAGE

Deterministic output tests

The deterministic generators — process compiler, IO linking, OB1 scaffold, type-conversion FCs — have unit tests in Vitest covering representative inputs. They run on every commit. Target coverage at MVP: 70% line coverage on the deterministic libraries, 0% on the AI-driven paths.

03 — SPECIFICATION

Structured logic checks

The Step 08 logic check — generated FB code validated against the FDS spec across ten categories — is itself a deterministic test layer. It catches the highest-impact category of error: AI generates code that compiles but doesn't behave per spec.

NOT TESTED AT MVP

- AI-generated outputs are not unit-tested. The AI is non-deterministic and asserting on its outputs costs more than it's worth at MVP scale. Engineer review is the gate.
- HMI generation — deferred to a future phase.
- Multi-engineer concurrent editing — the MVP user count makes this low-priority.

15 DELIVERY

Roadmap & milestones

Six phases over approximately six months of full-time solo development.

PHASE	DURATION	DELIVERABLES
00 · Foundation	Wks 1-4	Project skeleton, Supabase schema for shared concerns, auth, design profile management, reference library, Pattern Librarian skeleton
01 · FDS Builder MVP	Wks 5-10	Instrument register, hierarchy wizard, co-authoring (per-assembly), system orchestration, structured spec editor, DOCX export
02 · Forge core path	Wks 11-16	Wizard steps 01–05, hardware/IO config, design profile-driven generation, FDS handoff
03 · Forge generation	Wks 17-20	Steps 06–10 (device FBs, assembly FBs, logic check, matrix review, device code), Pattern Librarian wired in
04 · Forge process & test	Wks 21-24	Steps 11, 12, 14 (PLCSim tests, process code), HMI step stubbed, TIA export (Step 15)
05 · Internal pilot	Wks 25-28	Two real customer projects delivered through Forja; bug fixes; documentation; backup procedures

Milestone gates require engineer-developer sign-off before the next phase begins. The most likely points of slip are:

- End of Phase 01 — FDS Builder co-authoring is the most behaviourally complex module.
- The Phase 03-to-04 transition — process code generation has the most edge cases.

> REALISTIC DELIVERY RANGE
>
> 24 to 32 weeks for solo full-time work. The schedule above is the optimistic-but-honest case.

16 DELIVERY

Resource plan

People

- One full-time engineer-developer (Kasper) — design, build, deploy, document.
- Studio Principal — milestone reviews, strategic input, no operational involvement.

Operating costs · monthly · MVP scale

ITEM	ESTIMATE (AUD/MONTH)
Supabase Pro	$40
Anthropic API	$300-$800 (varies with usage)
Vercel · post-Stage 03	$30
Dropbox · already in place	–
Subtotal	$370-$870

Operating costs only; capital costs (development time) are absorbed internally.

Knowledge inputs

- Pac Technologies house style guide — existing.
- Two to three completed historical projects, used as test cases for the FDS Builder and Forge Wizard.
- Siemens manuals — Programming Style Guide V2.1, SCL reference, instruction reference.
- One or two customer projects for the internal pilot phase.

17 RISK

Risk register

#	RISK	LIKELY	IMPACT	MITIGATION
R-01	AI generates incorrect safety-critical logic that passes engineer review	Low	Very High	Engineer review is mandatory before any artefact reaches a customer site. Safety-classified specs trigger additional structured checks.
R-02	TIA Openness API breaks between versions	Medium	High	Bridge tested against each new TIA version before production use. Frontend tolerates bridge unavailability with read-only mode.
R-03	Solo developer bus factor — single point of knowledge failure	High	High	All architectural decisions documented inline and in module READMEs. Code is conventionally structured.
R-04	Engineer adoption resistance — habits resist new workflows	Medium	High	Internal pilot uses real customer projects, not toy examples. Tool must save time on day one.
R-05	Scope creep into HMI / vendor integrations	High	Medium	Out-of-scope items listed explicitly in Appendix B. Reintroduction requires written change request.
R-06	AI provider cost scales faster than expected	Low	Medium	Monthly monitoring with alert thresholds. Haiku used for cheap classification work; Sonnet reserved for generation.
R-07	Code quality regressions undetected without tests	Medium	High	Vitest test infrastructure in Phase 00 covers deterministic generators. Manual smoke-test checklist for AI-driven paths.
R-08	Customer-facing project quality issues attributable to AI-generated code	Low	Very High	All artefacts are reviewed and TIA-compiled before delivery. Customer-facing project quality is owned by the engineer, not the tool.
R-09	Database schema evolution fails to preserve existing data	Low	High	All migrations are forward-only and tested against a copy of production. Backups taken before every schema change.
R-10	Third-party vendor integration becomes urgent before architecturally addressed	Medium	Medium	Pattern: future-phase work, deliberately deferred. Manual workaround acceptable in the interim.

#	RISK	LIKELY	IMPACT	MITIGATION
R-11	AI provider rate limits or outages disrupt active engineering work	Medium	Low	Generations are resumable from the last completed step. Read-only access remains during outage.
R-12	Reference library content quality is poor — outdated or contradictory documents	Medium	Medium	Conflict detection runs across reference sources. Engineers can prioritise sources per project. Periodic library review every six months.

18 RISK

Assumptions & open questions

Assumptions

- The studio's two to three engineers are willing to use Forja on real projects from the pilot phase onward.
- The TIA Portal Openness API remains stable across V17–V20 for the duration of MVP development.
- Anthropic's API pricing remains within an order of magnitude of current rates.
- No regulatory regime — functional safety certification, customer-imposed code review process — blocks AI-generated code from reaching TIA Portal in customer projects.
- Pac Technologies has the right to use historical projects as training inputs to the pattern library and reference library.

Open questions

Require resolution before, or during, Phase 00.

01 **Pilot project selection.** Which two real customer projects will be delivered through Forja during the pilot phase? Choice affects which assembly templates are prioritised in V1 catalogue.

02 **Code review policy for generated artefacts.** Does AI-generated code require a second engineer's sign-off before TIA upload, or is the generating engineer's review sufficient?

03 **Library contribution policy.** Who can author new function block templates and reference library entries — any engineer, or only the principal? Affects governance design.

04 **Pattern approval policy.** Are patterns auto-approved by the engineer who generated them, or do they queue for principal review? Trade-off between learning velocity and quality control.

05 **Backup and disaster recovery scope.** What's the studio's tolerance for data loss? Affects backup frequency and retention policy.

06 **Failure-mode policy.** When Forja cannot handle a project segment — a third-party vendor controller — what's the documented fallback?

07 **External licensing direction.** What's the target window for considering external licensing — twelve months, twenty-four? Affects whether multi-tenant architecture should influence MVP design choices.

08 **Customer disclosure.** Do customers need to know AI was used in producing their project code? Affects contracts, liability, and marketing.

A APPENDIX

Glossary

Assembly	A coordinated group of devices working together — a conveyor, a lift table, a pusher.
Device	A single physical thing with IO signals — a motor, a limit switch, a solenoid.
DOL	Direct-on-line motor — uncontrolled start, contactor only.
FB	Function Block (Siemens). Reusable block of code with internal state, instantiated as instance DBs.
FC	Function (Siemens). Stateless block of code.
FDS	Functional Description Specification. The structured behavioural document produced by the FDS Builder.
HMI	Human-Machine Interface — the operator screens.
LAD	Ladder Diagram. One of the IEC 61131-3 PLC programming languages.
OB	Organization Block (Siemens). Top-level entry point.
PLC	Programmable Logic Controller.
PLCSim	Siemens' PLC simulator. Used for offline testing without physical hardware.
SCL	Structured Control Language. Pascal-like PLC language; one of the IEC 61131-3 standards.
Subsystem	A functional station within a machine — infeed conveyor station, hydraulic lift station.
TIA Portal	Siemens' integrated engineering environment for Siemens controllers and HMIs.
TIA Openness	Siemens' API for programmatic interaction with TIA Portal projects.
UDT	User-Defined Type. A struct-like typed record in TIA Portal.
VSD	Variable Speed Drive — controlled motor speed via drive electronics.

B APPENDIX

Not currently taking on

Out of scope for the MVP. Each item is recorded with the rationale for deferral and the trigger to revisit it.

ITEM	RATIONALE FOR DEFERRAL	TRIGGER TO RECONSIDER
HMI screen generation	Substantial workstream of its own; PLC-side work is the higher-leverage starting point	Two MVP customer projects delivered; HMI becomes the next bottleneck
KUKA / robot integration	Deep vendor-specific knowledge; prone to spec changes; would consume disproportionate MVP capacity	First customer project that requires it
ABB / non-Siemens drives	Same as above	First customer project that requires it
Multi-tenant external hosting	Architecturally significant; only justified once internal use is proven	External licensing decision made
Mobile clients	No use case identified for an engineer working away from a workstation	If field commissioning workflow emerges as a need
Live runtime telemetry	Outside Forja's design-time scope	Separate product investigation
Auto unit-test generation	Research-grade problem; immature tooling industry-wide	Industry-standard tools mature
Customer-facing portals	Out of scope for an internal tool	External licensing decision made

— END OF DOCUMENT —